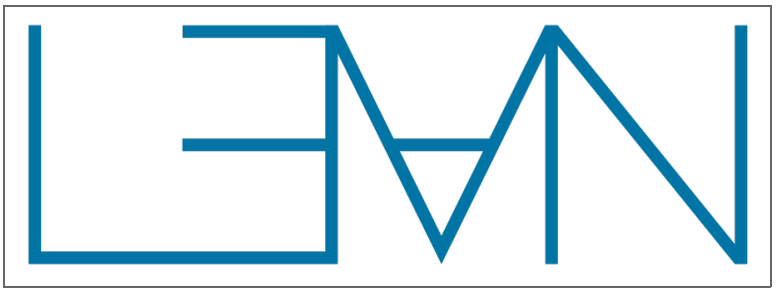


Proof Automation and AI

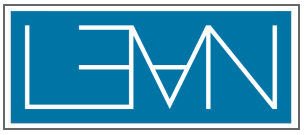
Lecture 3 of 4

Leo de Moura

Senior Principal Applied Scientist, AWS

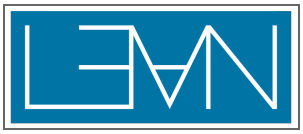
Chief Architect, Lean FRO

Marktoberdorf Summer School | August 2026



Today

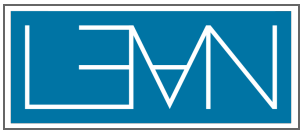
1. **`simp`** : the rewriting engine you have been using — how to use it well.
2. **`grind`** : congruence closure, E-matching, and theory solvers, native to dependent type theory.
3. **`bv_decide`** : SAT-based decisions for bit-level code, kernel-checked.
4. **AI**: what it already does with Lean, and why trust still comes from the kernel.



Do We Still Need Proof Automation?

> *"I thought AI would prove all theorems for us now."*

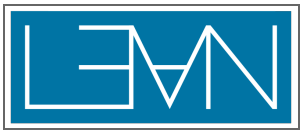
- Tactics are the **moves** of the game — for humans and for AI.
- Better tactics = shorter proofs.
- Shorter proofs = smaller search trees = more capable AI.
- Compact proofs = better training data.



`simp` : Rewriting with a Database

```
example (xs : List Nat) : (xs ++ []).map (· + 0) = xs := by
  simp
```

- `simp` rewrites with `@[simp]` lemmas, left to right, until no rule applies.
- Thousands of library lemmas are annotated; your own join with `@[simp]` or `simp [myLemma]` .
- `simp at h` rewrites a hypothesis; `simp_all` saturates hypotheses and goal together.



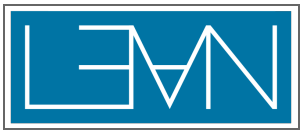
What Makes a Good `simp` Lemma

```
def double (n : Nat) : Nat := 2 * n

@[simp] theorem double_eq (n : Nat) : double n = 2 * n := rfl

theorem double_add (a b : Nat) :
    double (a + b) = double a + double b := by
  simp +arith
```

- Left side more complex than the right: rewriting must make progress toward a **normal form**.
- The set must be **confluent enough**: order should not matter.
- Recall Lecture 2: one lemma per helper function, stated in the normal form you want.



Conditional Rewriting

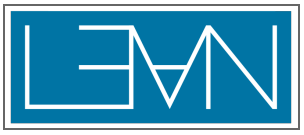
```
def safeDiv (a b : Nat) : Nat := if b = 0 then 0 else a / b

@[simp] theorem safeDiv_self (h : b ≠ 0) : safeDiv b b = 1 := by
  simp [safeDiv, h]
  exact Nat.div_self (Nat.pos_of_ne_zero h)

example (n : Nat) (h : n ≠ 0) : safeDiv n n + 1 = 2 := by
  simp [h]

example (n : Nat) (h : n > 5) : safeDiv n n + 1 = 2 := by
  simp (disch := grind)
```

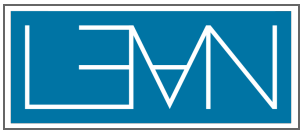
- `simp` lemmas may have hypotheses; `simp` discharges them recursively (with itself, and with the facts you pass).
- This is where rewriting starts needing **search** — the boundary where `grind` takes over.



Discovery: `simp?` , `apply?` , `exact?`

- `simp?` — closes the goal, then prints the minimal `simp only [...]` call. Use it, then paste the result: faster and more robust.
- `apply?` / `exact?` — search the library for lemmas matching the goal.
- **Loogle** — search by type shape from the browser or editor.

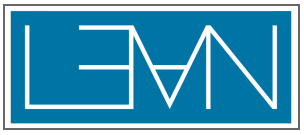
Automation is also for **finding** the lemma, not only for closing goals.



Where `simp` Stops

```
example (x y : Int) (h₁ : 2 * x + 3 * y = 7) (h₂ : 3 * x - y = 5) :
    x = 2 := by
  grind
```

- No rewrite rule solves a linear system: this needs **arithmetic reasoning**, not normalization.
- Equalities must also flow **between** hypotheses, through function applications, into case splits.
- That combination — rewriting + theories + propagation — is `grind` .

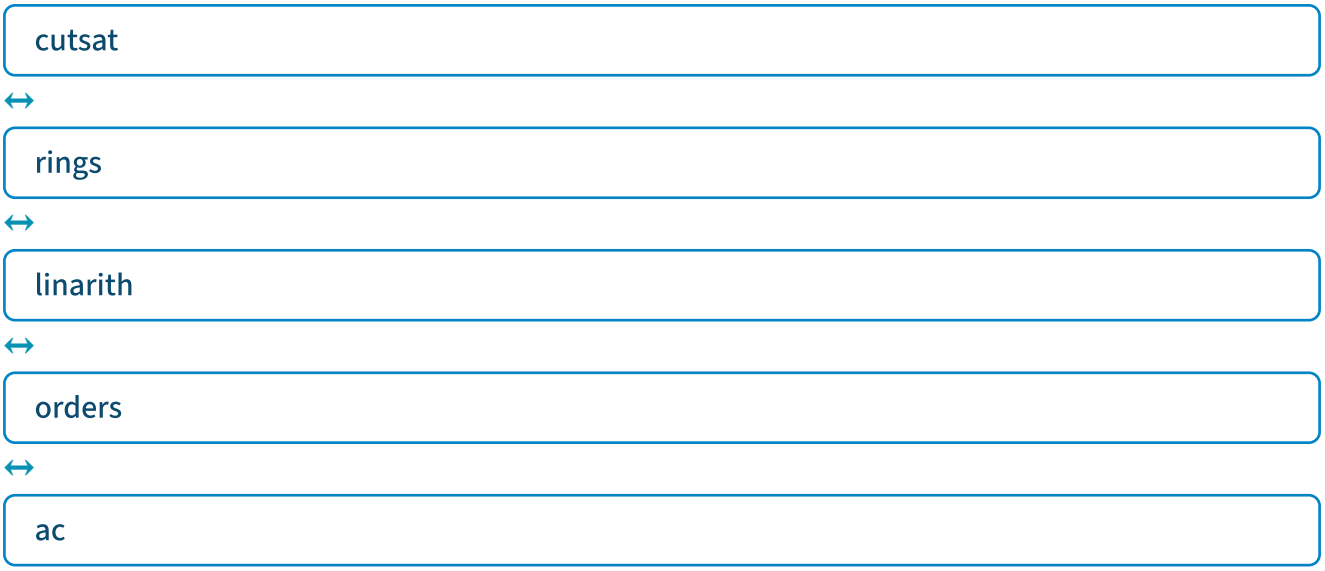


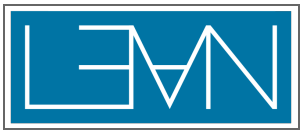
What is `grind` ?

New proof automation, shipped in Lean v4.22. Kim Morrison and me.

A **virtual whiteboard**, inspired by modern SMT solvers.

- Writes facts on the board. Merges equivalent terms.
- Cooperating engines: **congruence closure**, **E-matching**, **constraint propagation**, **guided case analysis**.
- Satellite theory solvers: **cutsat** (linear integer arithmetic), **commutative rings** (Gröbner bases), **linarith**, **AC**.
- **Native to dependent type theory.** No translation to first-order logic.
- Produces ordinary Lean proof terms. **Kernel-checkable.**
- **Reference manual**.

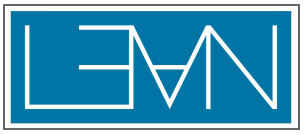

cutsat
↔
rings
↔
linarith
↔
orders
↔
ac



The Whiteboard: Congruence Closure

```
example (f : Nat → Nat) (a b c : Nat)
    (h₁ : a = b) (h₂ : f b = c) : f a = c := by
  grind
```

- The E-graph maintains equivalence classes of terms: `a = b` merges the classes of `a` and `b`.
- **Congruence**: equal arguments give equal applications, so `f a` and `f b` merge too. No rewriting happens — this is union-find.
- Every other engine reads from and writes to this board.

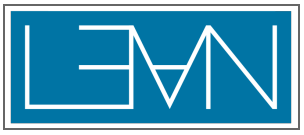


E-matching: Instantiating Lemmas

```
@[grind =] theorem fg {x} : f (g x) = x := by
  unfold f g; grind

example {a b c} : f a = b → a = g c → b = c := by
  grind
```

- Library authors annotate theorems; `grind` instantiates them by **E-matching**: pattern matching **modulo the board's equalities**.
- `[grind =]` uses the left-hand side as the pattern. Note `f (g c)` appears **nowhere** in the goal: the board knows `a = g c`, so `f a` matches `f (g ?x)`, the instance `f (g c) = c` lands on the board, and congruence chains `b = f a = f (g c) = c`.
- This is what makes annotation-driven automation **robust**: lemmas fire when the **equalities** match, not the syntax.



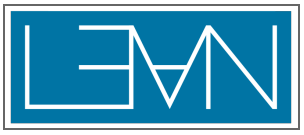
Annotating Your Own Library

```
@[grind =] theorem State.get_set_same (σ : State) (x : String) (v : Int)
    (σ.set x v).get x = v := by
  simp [State.get, State.set]

@[grind =] theorem State.get_set_ne (σ : State) (x y : String) (v : Int)
    (h : y ≠ x) : (σ.set x v).get y = σ.get y := by
  simp [State.get, State.set, h]

example (σ : State) (h : σ.get "y" = 5) :
    ((σ.set "x" 1).set "z" 2).get "y" = 5 := by
  grind
```

The `State` API is Lecture 2's; the two annotations make `grind` an expert in it — Lecture 4 runs on exactly these two lemmas.



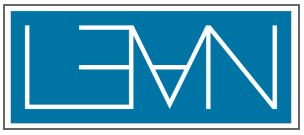
Forward Rules: `[grind →]`

```
def divides (a b : Nat) : Prop := ∃ k, b = a * k

@[grind →] theorem divides_trans (h₁ : divides a b) (h₂ : divides b c) :
    divides a c := by
  obtain ⟨k₁, rfl⟩ := h₁
  obtain ⟨k₂, rfl⟩ := h₂
  exact ⟨k₁ * k₂, Nat.mul_assoc a k₁ k₂⟩

example (h₁ : divides a b) (h₂ : divides b c) (h₃ : divides c d) :
    divides a d := by
  grind
```

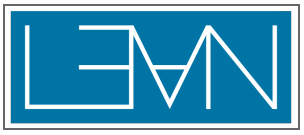
- `[grind →]` marks a **forward** rule: the patterns come from the premises — when matching facts are on the board, the conclusion is added. (`[grind ←]` is the dual: patterns from the conclusion.)
- Transitivity is the canonical case: two firings chain the three hypotheses into `divides a d`.
- Forward chains can grow; `grind` bounds instantiation rounds, and its diagnostics list every instance created.



Satellite Solvers, Activated by Type Classes

Theory solvers engage **automatically** when the type classes are present:

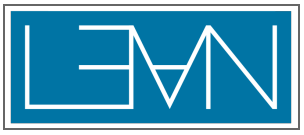
- **cutsat** — linear integer arithmetic; `Int`, `Nat`, `Int32`, `BitVec n`, `Fin n`.
- **Ring** — `CommRing`, `Field`, `IsCharP`: Gröbner-basis reasoning.
- **linarith** — ordered modules: linear arithmetic over ordered fields.
- **AC** — any associative-commutative operator.



Linear Integer Arithmetic

```
example (x y : Int)
    : 27 ≤ 11 * x + 13 * y →
      11 * x + 13 * y ≤ 45 →
      -10 ≤ 7 * x - 9 * y →
      7 * x - 9 * y ≤ 4 → False := by
  grind
```

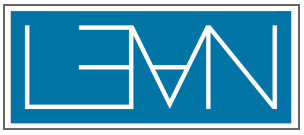
- `cutsat` : a complete decision procedure for linear integer arithmetic — the workhorse for indices, sizes, and bounds in program verification.
- `lia` (Lecture 2) is `cutsat` without the rest of `grind` .



Commutative Rings

```
example (x : BitVec 8) : (x - 16) * (x + 16) = x ^ 2 := by
  grind
```

- `BitVec 8` is a commutative ring of characteristic 256 — the ring solver proves this by normalization; no bitvector theory needed.
- The same solver handles polynomial identities over `Int`, `Rat`, and any `CommRing`.



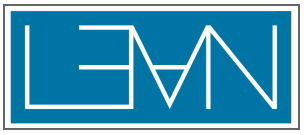
Theory Combination

```
example [CommRing α] [NoNatZeroDivisors α]
    (a b c : α) (f : α → Nat)
    : a + b + c = 3 →
      a ^ 2 + b ^ 2 + c ^ 2 = 5 →
      a ^ 3 + b ^ 3 + c ^ 3 = 7 →
      f (a ^ 4 + b ^ 4) + f (9 - c ^ 4) ≠ 1 := by
  grind
```

Three solvers meet at the E-graph:

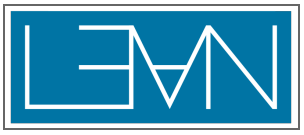
- **Ring solver** derives `a^4 + b^4 = 9 - c^4`.
- **Congruence closure** lifts it to `f (a^4 + b^4) = f (9 - c^4)`.
- **Linear integer arithmetic** closes `2 * f (9 - c^4) ≠ 1`.

The Nelson–Oppen playbook, inside dependent type theory — no SMT translation layer.



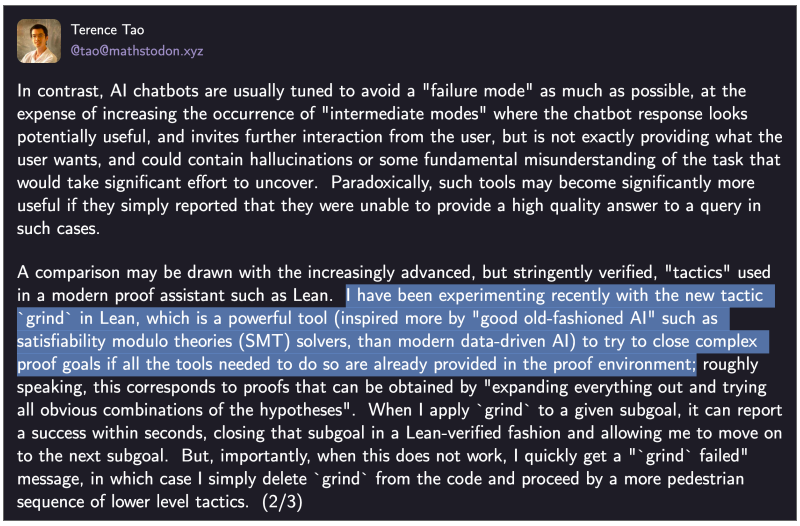
Guided Case Analysis

- `grind` splits on `match` expressions, `if` s, and annotated disjunctions — with heuristics to avoid explosion.
- Recall the ladder from Lecture 2: `mkAdd.eq_def` handed `grind` the match equations, and it did the case analysis itself.
- Case analysis is where automation costs blow up; controlling it is half the engineering.



`grind` in Practice

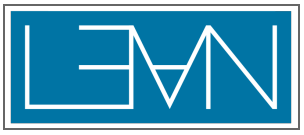
- **5,000+ `grind` calls in Mathlib.**
- Used daily on goals from algebra, program verification, and combinatorics.



Terence Tao
@tao@mathstodon.xyz

In contrast, AI chatbots are usually tuned to avoid a "failure mode" as much as possible, at the expense of increasing the occurrence of "intermediate modes" where the chatbot response looks potentially useful, and invites further interaction from the user, but is not exactly providing what the user wants, and could contain hallucinations or some fundamental misunderstanding of the task that would take significant effort to uncover. Paradoxically, such tools may become significantly more useful if they simply reported that they were unable to provide a high quality answer to a query in such cases.

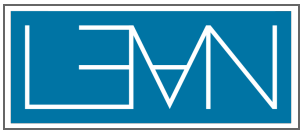
A comparison may be drawn with the increasingly advanced, but stringently verified, "tactics" used in a modern proof assistant such as Lean. I have been experimenting recently with the new tactic `grind` in Lean, which is a powerful tool (inspired more by "good old-fashioned AI" such as satisfiability modulo theories (SMT) solvers, than modern data-driven AI) to try to close complex proof goals if all the tools needed to do so are already provided in the proof environment; roughly speaking, this corresponds to proofs that can be obtained by "expanding everything out and trying all obvious combinations of the hypotheses". When I apply `grind` to a given subgoal, it can report a success within seconds, closing that subgoal in a Lean-verified fashion and allowing me to move on to the next subgoal. But, importantly, when this does not work, I quickly get a "`grind` failed" message, in which case I simply delete `grind` from the code and proceed by a more pedestrian sequence of lower level tactics. (2/3)



What `grind` Is Not

- Not designed for combinatorially explosive search spaces.
- Not a translation to an external SMT solver: no encoding gap, no reconstruction gap.
- Not a nonlinear-arithmetic oracle: the ring solver normalizes polynomial **equalities**; hard nonlinear **inequalities** remain hard.
- A workhorse with a legible design — when it fails, you can see why.

For massive case-analysis, use `bv_decide` .



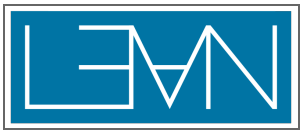
When `grind` Fails

```
example (as bs cs : Array α) (v : α) (i : Nat)
        (h₁ : i < as.size)
        (h₂ : bs = as.set i v)
        (h₅ : j < bs.size)
        (h₆ : j < as.size)
        : bs[j] = as[j] := by
  grind
```

- On failure, `grind` prints its board: the facts it knew, the classes it built, the instances it tried.
- Read the board, find what is missing — a fact, an annotation, or, as here, a hypothesis: nothing rules out `j = i`, where the claim is false. The loop is: **fail, read, fix, succeed**.
- Automation you can inspect and extend — not a black box.

Bit-Level Verification

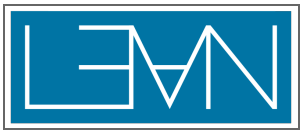
Hardware, cryptography, systems code.



`BitVec` and the Problems That Need It

```
#check (0xFF : BitVec 8)
  255 : BitVec 8
#eval (200 : BitVec 8) + 100
  44#8
#eval (1 : BitVec 8) <<< 7
  128#8
```

- Fixed-width machine arithmetic: overflow, shifts, masks — where hand proofs are least pleasant and bugs are most common.
- Compilers, crypto kernels, hardware models: all live here.
- `grind` 's ring solver handles some of it; complete answers need a **decision procedure**.

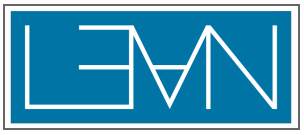


`bv_decide` : SAT with a Verified Checker

```
example (x y : BitVec 32) : x ^^^ y ^^^ x = y := by
  bv_decide

example (x : BitVec 32) : x &&& (x - 1) = x - (x &&& -x) := by
  bv_decide
```

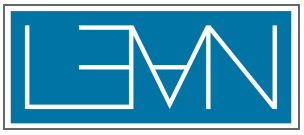
- Bit-blasts the goal to SAT, runs CaDiCaL, and replays the solver's **LRAT certificate** through a checker **verified in Lean**.
- The SAT solver is not trusted; the kernel still checks everything.



Counterexamples

```
example (x : BitVec 8) : x &&& (x - 1) = x - 1 := by
  bv_decide
```

- A decision procedure answers **both ways**: proof, or counterexample.
- `x = 0 : 0 &&& 255 = 0`, but `0 - 1 = 255`. The "identity" is false.
- Cheaper than a failed proof attempt: try the theorem before investing in it. (Shown as output — a failing tactic would, correctly, fail this slide's build.)



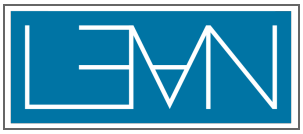
`popcount` , from Hacker's Delight

```
def popSpec (x : BitVec 32) : BitVec 32 :=
  go 32 x
where
  go : Nat → BitVec 32 → BitVec 32
    | 0, _ => 0
    | n + 1, x => (x &&& 1) + go n (x >>> 1)

def popcount (x : BitVec 32) : BitVec 32 :=
  let x := x - ((x >>> 1) &&& 0x55555555)
  let x := (x &&& 0x33333333) + ((x >>> 2) &&& 0x33333333)
  let x := (x + (x >>> 4)) &&& 0x0F0F0F0F
  let x := x + (x >>> 8)
  let x := x + (x >>> 16)
  x &&& 0x0000003F

theorem popcount_correct (x : BitVec 32) : popcount x = popSpec x := by
  simp [popcount, popSpec, popSpec.go]
  bv_decide
```

The bit-twiddling classic, verified against the obvious specification in two tactic lines.

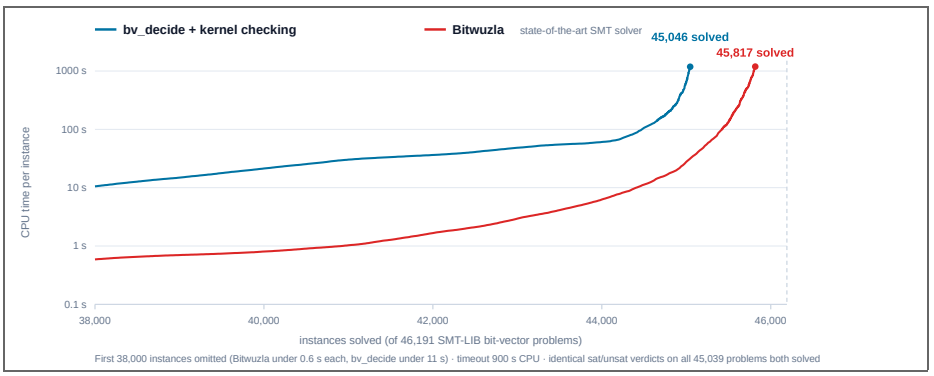


How Far Does It Scale?

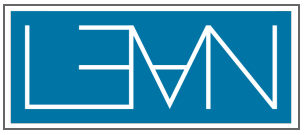
Benchmarked against Bitwuzla on **46,191** SMT-LIB bit-vector problems:

- **`bv_decide` solves 45,046 (97.5%) — kernel checking included.** Bitwuzla solves 45,817.
- **Identical sat/unsat verdicts on every problem both solved.**
- Total CPU time within 2.3× of Bitwuzla.

A verified pipeline, competitive with an unverified state-of-the-art solver.



AI and Lean



The IMO Milestones

Every medal-level IMO AI with formal proofs uses Lean.

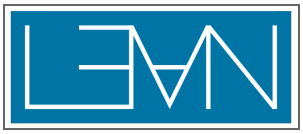
- **AlphaProof** (Google DeepMind) — silver medal, IMO 2024
- **Aristotle** (Harmonic) — gold medal, IMO 2025
- **Seed Prover** (ByteDance) — silver medal, IMO 2025

In 2019, the IMO was posed as a **Grand Challenge** for AI. Six years later, three independent systems have medal-level results.

Moves played by **AlphaProof**:

```
simp_all [Finset.sum_range_id]
zify [*] at *
norm_num at *
nlinarith [(by norm_cast : (c:ℝ) ≥ A*(l-⌊_⌋)+⌊_⌋+1),
           Int.floor_lex, Int.lt_floor_add_one x]
```

- The most advanced AI relies on the same tactics we use every day.
- When `grind` closes a goal in one step instead of fifty, the AI search tree shrinks accordingly.
- Better tactics make more capable AI — automation did not become obsolete; it became infrastructure.



"Vibe Proving"

Alexeev & Mixon resolved a **$1000 Erdős prize problem**.

> *"We used ChatGPT to vibe code a Lean proof." — Alexeev & Mixon*

The proof is machine-checked. **Paper**

Forbidden Sidon subsets of perfect difference sets, featuring a human-assisted proof

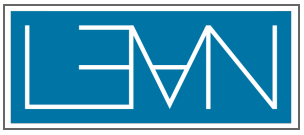
Boris Alexeev ChatGPT* Lean† Dustin G. Mixon‡§

Abstract

We resolve a $1000 Erdős prize problem, complete with formal verification generated by a large language model.

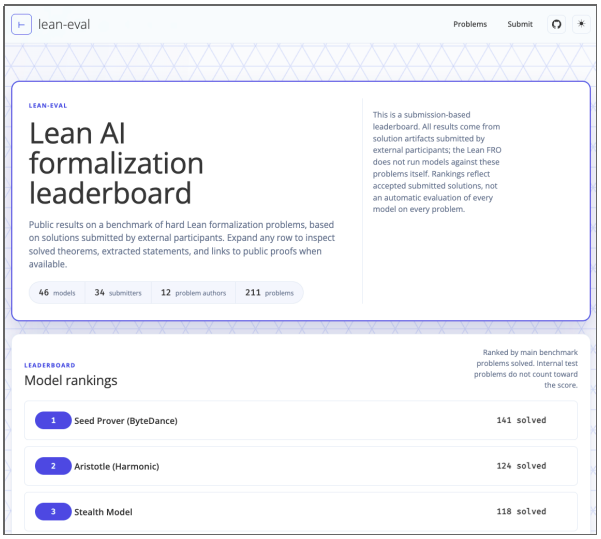
In over a dozen papers, beginning in 1976 and spanning two decades, Paul Erdős repeatedly posed one of his "favourite" conjectures: every finite Sidon set can be extended to a finite perfect difference set. We establish that $\{1, 2, 4, 8, 13\}$ is a counterexample to this conjecture.

During the preparation of this paper, we discovered that although this problem was presumed to be open for half a century, Marshall Hall, Jr. published a different counterexample three decades *before* Erdős first posed the problem. With a healthy skepticism of this apparent oversight, and out of an abundance of caution, we used ChatGPT to vibe code a Lean proof of both Hall's and our counterexamples.



Lean Eval

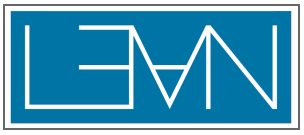
- **Lean AI formalization leaderboard**
- Public results on a benchmark of **hard** Lean formalization problems.
- Submission-based leaderboard.
- **Erdős's unit-distance conjecture is one of the problems**.
  - OpenAI showed this 80-year-old conjecture to be false.
  - Boris Alexeev submitted a ~1.2M-line Lean formal proof.
  - **"Human mathematicians are**



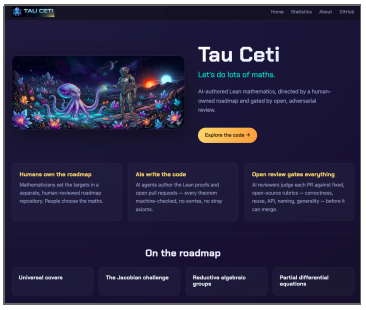
being outcounterexampled" — Kevin Buzzard

Resume presentation

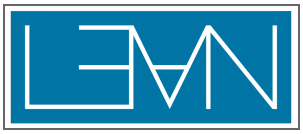
- **Feit–Thompson odd-order theorem** — 4 submissions
- **Jacobian of a compact Riemann surface (Buzzard challenge)** — 4 submissions



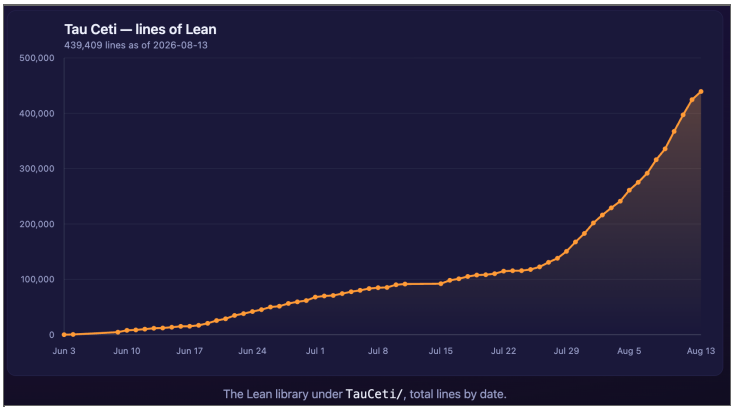
Tau Ceti: AI-Authored Lean Mathematics



- **AI-authored Lean mathematics, directed by a human-owned roadmap and gated by open, adversarial review.**
- Humans own the roadmap: mathematicians choose the targets.
- AIs write and review the code.
- On the roadmap: universal covers, the Jacobian challenge, reductive algebraic groups, PDEs.
- **taucetiproject.github.io/TauCeti**

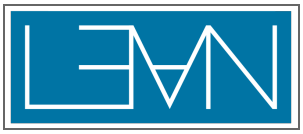


Tau Ceti: Growth

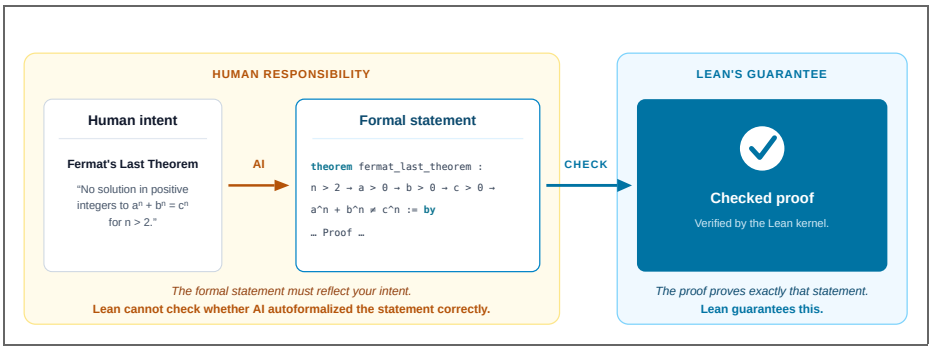


To turn any computer into a Tau Ceti contributor:

```
uv tool install git+https://github.com/kim-em/TauCetiWorker
tauceti work --loop
```

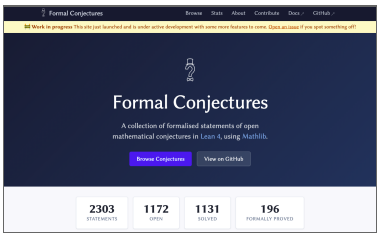


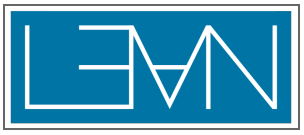
The Statement / Specification Is Your Responsibility



Formal Conjectures

(Google DeepMind): curated, human-verified formal statements of open problems.





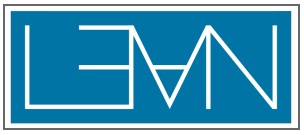
Certified Computer Algebra: Hex

Hex: computational algebra for Lean — LLL lattice reduction, and now integer polynomial factorization.

```
example : Irreducible (X ^ 4 + 8 * X + 12 : Polynomial ℤ) := by
  irreducibility
```

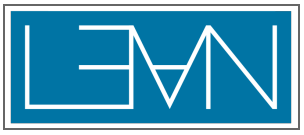
- Berlekamp, Hensel lifting, Berlekamp–Zassenhaus, with van Hoeij's knapsack reconstruction.
- **The van Hoeij algorithm had never been formally verified before.**
- Consistently faster than Isabelle's verified factorization; about 5× slower than FLINT (unverified state of the art).

Kim Morrison, **August 2026**.



Agents and Verified Software

- **lean-zip** (Lecture 1): AI agents optimized the code **autonomously** — the round-trip theorem stood in for human review of each change.
- **Radix**: 10 AI agents built a verified DSL in a weekend — 52 theorems, ~7,400 lines, 0 `sorry`, 5 verified optimization passes. **github.com/leodemoura/RadixExperiment**
- The pattern: humans own specifications; agents iterate on code and proofs.



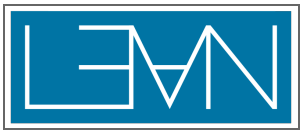
What AI Is Good At (and Not)

Good, today:

- writing complex formal proofs
- explaining formal proofs
- metaprogramming
- isolating and diagnosing bugs; optimizing code; translating code

Not good, today:

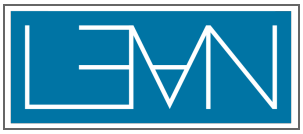
- system-level design; novel abstractions
- long-running context; knowing when a specification is **wrong**.



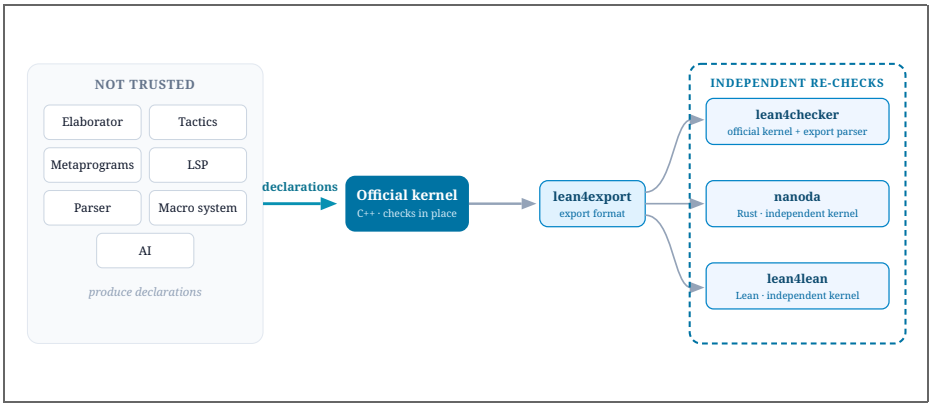
AI Will Exploit Any Unsoundness

> *"It's really important with these formal proof assistants that there are no backdoors or exploits you can use to somehow get your certified proof without actually proving it, because reinforcement learning is just so good at finding these backdoors." — Terence Tao*

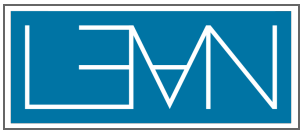
- RL systems optimize the checker, not the mathematics.
- Tools with a large trusted base are a liability in the AI era.
- This is why the kernel architecture from Lecture 1 matters **more** now, not less.



Layered Trust



- **Everyday**: the file checks; `#print axioms` shows dependencies.
- **Replay**: `lean4checker` re-runs the kernel on compiled output.
- **Gold standard**: export the proof; recheck with **independent kernels** (Rust, Lean, C++ implementations) at **arena.lean-lang.org**.
- **Comparator**: sandboxed judging for AI-submitted proofs — defeats metaprogramming exploits.

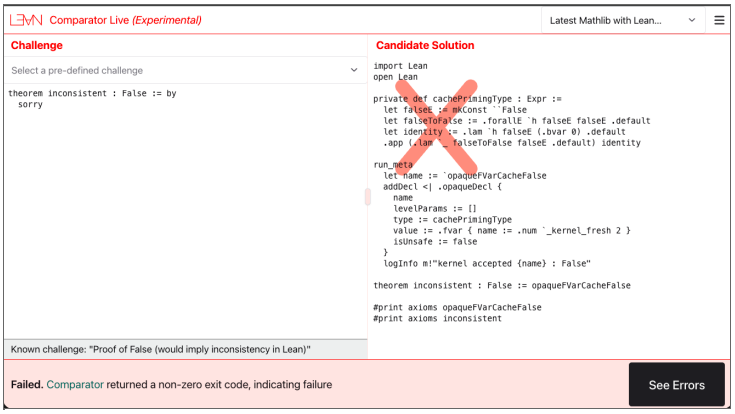


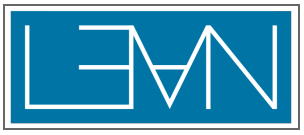
Comparator in Action

A challenge asks for a proof of `False`.

A candidate tries to smuggle one past the kernel with a metaprogramming trick that exploits a missing check in the official kernel — a real **GitHub issue**.

Comparator rejects it. The proof is exported and rechecked independently; **Nanoda** (Rust) and **Lean4Lean** reject it too.



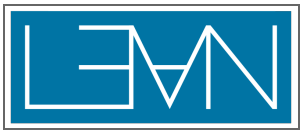


Doubling Down on Trust

On July 25, an AI managed to construct a proof of `False` that exploited a bug in the official kernel, and completely different bug in nanoda, the main external kernel.
Our **postmortem** for additional details.

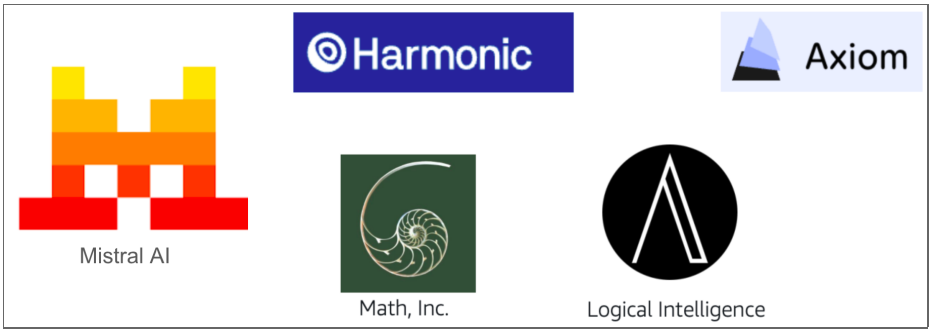
More kernels, verified kernels:

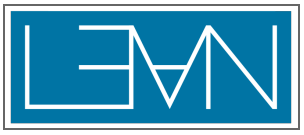
- `lake` commands to check Lean developments using `comparator`, `nanoda`, etc.
- We are collaborating with AI teams that have access to AI with cybersecurity capabilities.
- We are actively hardening kernel invariants.
- We are considering using the Lean 3 approach as a sanity check. In Lean 3, we compiled nested/mutual inductives into simple ones, provided definitions for the constructors and recursors, and proved that the reduction rules were propositionally true.
- We are approaching people to implement new kernels, and we are willing to fund them.
- We want to see `lean4lean` fully verified.



AI Startups on Lean

- **Leanstral** (Mistral): open-source code agent designed for Lean 4.
- **Axiom**: solved 12/12 problems on Putnam 2025.
- **DeepSeek Prover-V2**: 88.9% on miniF2F. Open source, 671B parameters.
- **Harmonic**: built Aristotle — gold medal, IMO 2025.

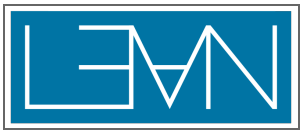




Exercises

`Exercises/Lecture3.lean` — the proofs should be **short**:

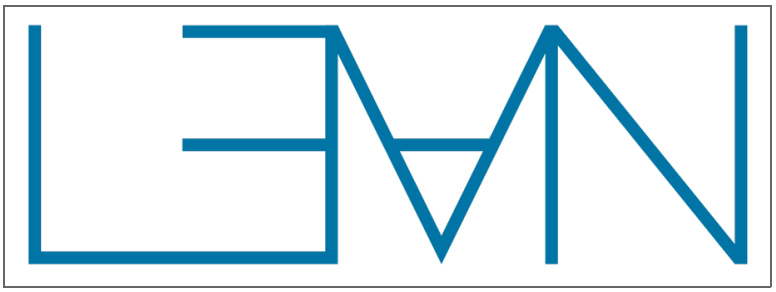
1. `simp` lemma design.
2. `grind` : congruence, arithmetic, and theory cooperation — prove each by hand first, then replace with `grind` .
3. `[grind =]` annotation: make E-matching fire modulo equalities.
4. The ladder, on the optimizer from Lecture 2.
5. `bv_decide` : xor tricks, two's complement, a buggy xor-swap to fix, branch-free `abs` .



Summary

- `simp` : rewriting to normal forms; design your lemmas.
- `grind` : an E-graph where congruence, E-matching, and theory solvers cooperate — extensible with two-line annotations.
- `bv_decide` : complete for bitvectors, certificate-checked, competitive.
- AI uses these same tools, at scale; the kernel remains the arbiter.

Next lecture: Hoare logic, a verified verification condition generator, and the engineering that makes verification scale — because once AI writes proofs, the limit is platform throughput.



Thank You

Leo de Moura

lean-lang.org

leanprover.zulipchat.com

leodemoura.github.io

Marktoberdorf Summer School | August 2026

Speaker notes